LABORATORY RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Branch: CSM
Section: B
Task No.: 4
Student Name: M. Niharika
Roll No.: A24126552097

1. TITLE

JavaScript Fundamentals – Arrays, Functions, and Object-Oriented Programming Concepts

2. AIM

To understand and implement JavaScript array operations, different types of functions (regular, arrow, and anonymous functions), callback functions, and object-oriented programming concepts such as classes, inheritance, and encapsulation using getters and setters.

3. OBJECTIVE

The objectives of this experiment are:

- To create and initialise an array in JavaScript.
- To access, modify, and traverse array elements using indices and loops.
- To use built-in array methods such as push() and pop().
- To define and invoke functions using single and multiple parameters.
- To differentiate between function declarations, arrow functions, and anonymous functions.
- To implement callback functions using named functions, anonymous functions, and arrow functions.
- To understand asynchronous behaviour using setTimeout().
- To create classes using ES6 class syntax with constructors and methods.
- To implement inheritance between classes using the extends keyword.
- To implement encapsulation using private class fields (#) along with getter and setter methods.

4. THEORY

An array in JavaScript is an ordered, resizable collection of values that can be accessed using zero-based numeric indices. Arrays expose a length property and built-in methods such as push()

(adds an element to the end) and pop() (removes the last element), which make it easy to manage dynamic data without manually resizing memory.

A function is a reusable block of code that can accept parameters and return a value. JavaScript supports several ways of writing functions: function declarations (hoisted, named), arrow functions (concise syntax, no own "this" binding), and anonymous functions (unnamed, often used inline). A callback is simply a function passed as an argument to another function, to be "called back" once some work is done — this is the basis of asynchronous operations such as setTimeout(), which schedules a callback to run after a delay without blocking the rest of the program.

Object-Oriented Programming (OOP) in JavaScript is implemented using the class keyword, introduced in ES6 as syntactic sugar over prototypal inheritance. A class groups related data (fields) and behaviour (methods) together, and objects are created from a class using the new keyword and a constructor. Inheritance lets one class (a subclass) reuse and extend the behaviour of another (a superclass) using the extends keyword, so a Dog can inherit eat() from Animal while adding its own bark() method. Encapsulation restricts direct access to internal state: prefixing a field with # makes it a private field, accessible only from within the class, while get and set accessor methods provide controlled, validated read and write access to that private state from outside the class.

5. ALGORITHM / PROCEDURE

1. Declare an array of numbers and print it using console.log().
2. Read and display the array's length property.
3. Access and display the first and last elements using their indices.
4. Modify an existing element by assigning a new value at a given index.
5. Use push() to add a new element to the end of the array, and pop() to remove the last element.
6. Loop through the array with a for loop and print each element.
7. Define a function using a function declaration and call it with an argument.
8. Define a function that accepts multiple parameters and returns their computed result.
9. Define arrow functions with zero, one, and multiple parameters, and invoke each.
10. Define an anonymous function stored in a const variable and invoke it.
11. Write a function that accepts another function as a callback parameter, and pass a named function, an anonymous function, and an arrow function as the callback in turn.
12. Use setTimeout() to schedule a callback to run after a delay.
13. Define a Student class with a constructor and a display() method, then create and use an instance.
14. Define an Animal class and a Dog class that extends it, then call an inherited method and the subclass's own method.
15. Define a Bank class with a private #balance field and public getter/setter methods, and test valid and invalid updates.
16. Define an alternate Bank class that initialises #balance through the constructor and exposes deposit() and withdraw() methods, then test a deposit and a withdrawal.

6. PROGRAM

Program 1: Array Operations

```
// Create an array
let numbers = [10, 20, 30, 40, 50];

console.log("Original Array:", numbers);

// 1. Length
console.log("Length:", numbers.length);

// 2. Access Elements
console.log("First Element:", numbers[0]);
console.log("Last Element:", numbers[numbers.length - 1]);

// 3. Modify Element
numbers[1] = 25;
console.log("After Modification:", numbers);

// 4. push()
numbers.push(60);
console.log("After push:", numbers);

// 5. pop()
numbers.pop();
console.log("After pop:", numbers);

// display array elements
console.log("Array Elements:");
for (let i = 0; i < numbers.length; i++) {
   console.log(numbers[i]);
}
```

Program 2: Functions – Declarations, Arrow Functions, and Callbacks

```
// Function declaration
function greet(name) {
   return "hello " + name;
}
console.log(greet("alice"));

// function with multiple parameters
function add(a, b) {
   return a + b;
}
console.log(add(10, 30));
```

```
// arrow function without parameters
const greetArrow = () => {
   console.log("Hello World");
};
greetArrow();

// arrow function with 1 parameter
const square = num => num * num;
console.log(`Square of ${5} is ${square(5)}`);

// arrow function with multiple parameters
const addArrow = (a, b) => {
   return a + b;
};
console.log(`Sum of 10 & 20 is ${addArrow(10, 20)}`);

// anonymous function
const greets = function (name) {
   return "Hello " + name;
};
console.log(greets("bob"));

// callback functions
function hello(name, callback) {
   console.log("hello " + name);
   callback();
}
function bye() {
   console.log("Bye!");
}
hello("John", bye);

// callback using anonymous function
function greetings(name, callback) {
   console.log("Hello " + name);
   callback();
}
greetings("lilly", function () {
   console.log("Goodbye!");
});

// callback using arrow function
function good(name, callback) {
   console.log("Hello " + name);
   callback();
```

```
}
good("John", () => {
   console.log("Goodbye!");
});

// setTimeout callback
setTimeout(function () {
   console.log("Hello after 2 seconds");
}, 2000);
```

Program 3: Classes, Inheritance, and Encapsulation

```
class Student {

   constructor(name, age) {
      this.name = name;
      this.age = age;
   }

   display() {
      console.log("Name:", this.name);
      console.log("Age:", this.age);
   }
}

let s1 = new Student("Snoopy", 2);
s1.display();


class Animal {
   eat() {
      console.log("Eating");
   }
}

class Dog extends Animal {
   bark() {
      console.log("Barking");
   }
}

const d = new Dog();
d.eat();
d.bark();


class Bank {
```

```
    #balance = 10000;

    // Getter method
    get balance() {
        return this.#balance;
    }

    // Setter method
    set balance(amount) {
        if (amount >= 0) {
            this.#balance = amount;
        } else {
            console.log("Balance cannot be negative.");
        }
    }
}

let account = new Bank();

// Using getter
console.log("Initial Balance:", account.balance);

// Using setter
account.balance = 15000;
console.log("Updated Balance:", account.balance);

// Invalid update
account.balance = -500;
```

Program 4: Bank Class with Constructor-Initialised Balance

```
class Bank {

    #balance;

    constructor(balance) {
        this.#balance = balance;
    }

    get balance() {
        return this.#balance;
    }

    deposit(amount) {
        if (amount > 0) {
            this.#balance += amount;
        }
    }
```

```
    withdraw(amount) {
        if (amount <= this.#balance) {
            this.#balance -= amount;
        } else {
            console.log("Insufficient Balance");
        }
    }
}

let account = new Bank(10000);

account.deposit(5000);
account.withdraw(3000);

console.log("Balance:", account.balance);
```

7. CODE EXPLANATION

Code	Explanation
let numbers = [...]	Declares and initialises an array literal holding numeric values.
numbers.length	Returns the number of elements currently in the array.
numbers[0], numbers[n-1]	Accesses the first and last elements using zero-based indexing.
numbers[1] = 25;	Modifies the element at index 1 by direct assignment.
push() / pop()	Adds an element to the end of the array / removes the last element.
for (let i = 0; ...)	Iterates over the array and prints each element in turn.
function greet(name) {...}	A function declaration — hoisted, invoked by name with one argument.
function add(a, b) {...}	A function that accepts multiple parameters and returns their sum.
const greetArrow = () => {...}	An arrow function with no parameters, assigned to a constant.
const square = num => num * num;	A concise arrow function with a single parameter and implicit return.
const addArrow = (a, b) => {...}	An arrow function with multiple parameters and an explicit return.

Code	Explanation
const greets = function (name) {...}	An anonymous function expression stored in a variable.
function hello(name, callback) {...}	Accepts a function as a parameter and invokes it as a callback.
hello("John", bye);	Passes an existing named function (bye) as the callback.
greetings("lilly", function () {...});	Passes an anonymous function directly as the callback.
good("John", () => {...});	Passes an arrow function directly as the callback.
setTimeout(function () {...}, 2000);	Schedules the callback to run once, after a 2000 ms delay, without blocking the script.
class Student {...}	Defines a class with a constructor to set fields and a method to display them.
class Dog extends Animal {...}	Inherits Animal's eat() method and adds its own bark() method.
#balance	A private class field, accessible only from within the Bank class.
get balance() / set balance(amount)	Getter/setter pair providing controlled, validated read and write access to #balance.
deposit(amount) / withdraw(amount)	Public methods that safely increase or decrease the private balance, with a check for insufficient funds.

8. TEST CASES AND EXECUTION DATA

Each program was run in the Node.js console and the browser developer console, and the printed output was compared against the expected result.

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Run Program 1 and log the original array	Array [10, 20, 30, 40, 50] is printed	Array printed correctly	Pass
TC-02	Log numbers.length	Length 5 is printed	5 printed	Pass
TC-03	Access numbers[0] and last element	First = 10, Last = 50	First = 10, Last = 50	Pass
TC-04	Modify numbers[1] = 25	Array becomes [10, 25, 30, 40, 50]	Array updated correctly	Pass

Test Case	Action	Expected Result	Actual Result	Status
TC-05	Call numbers.push(60)	60 added to end of array	Array ends in 60	Pass
TC-06	Call numbers.pop()	Last element (60) removed	60 removed	Pass
TC-07	Call greet("alice")	"hello alice" is returned and printed	"hello alice" printed	Pass
TC-08	Call add(10, 30)	40 is returned and printed	40 printed	Pass
TC-09	Call square(5)	"Square of 5 is 25" printed	Correct string printed	Pass
TC-10	Call hello("John", bye)	"hello John" then "Bye!" printed	Both lines printed in order	Pass
TC-11	Call good("John", arrowFn)	"Hello John" then "Goodbye!" printed	Both lines printed in order	Pass
TC-12	Wait 2 seconds after setTimeout call	"Hello after 2 seconds" printed after delay	Message printed after ~2s	Pass
TC-13	Create new Student("Snoopy", 2) and call display()	Name and age printed to console	Name: Snoopy, Age: 2 printed	Pass
TC-14	Call d.eat() and d.bark() on a Dog instance	"Eating" then "Barking" printed	Both inherited and own methods worked	Pass
TC-15	Read account.balance (getter)	Initial balance 10000 printed	10000 printed	Pass
TC-16	Set account.balance = 15000	Balance updates to 15000	Updated balance 15000 printed	Pass
TC-17	Set account.balance = -500	Setter rejects it and prints a warning	"Balance cannot be negative." printed	Pass
TC-18	deposit(5000) then withdraw(3000) on Bank(10000)	Final balance = 12000	12000 printed	Pass

9. OUTPUT

Output 1: Array Operations – Console Output

```
Original Array: [ 10, 20, 30, 40, 50 ]
Length: 5
First Element: 10
```

```
Last Element: 50
After Modification: [ 10, 25, 30, 40, 50 ]
After push: [ 10, 25, 30, 40, 50, 60 ]
After pop: [ 10, 25, 30, 40, 50 ]
Array Elements:
10
25
30
40
50
```

Output 2: Functions and Callbacks – Console Output

```
hello alice
40
Hello World
Square of 5 is 25
Sum of 10 & 20 is 30
Hello bob
hello John
Bye!
Hello lilly
Goodbye!
Hello John
Goodbye!
Hello after 2 seconds   (printed after a 2 second delay)
```

Output 3: Classes, Inheritance, and Encapsulation – Console Output

```
Name: Snoopy
Age: 2
Eating
Barking
Initial Balance: 10000
Updated Balance: 15000
Balance cannot be negative.
```

Output 4: Bank Class (Deposit/Withdraw) – Console Output

```
Balance: 12000
```

10. RESULT

Thus, JavaScript array operations, various forms of functions (declarations, arrow functions, anonymous functions, and callbacks), and object-oriented programming concepts (classes, inheritance, and encapsulation using private fields with getters and setters) were successfully implemented and verified. All programs executed as expected, and the outputs were confirmed to match the expected results in each test case.